ESD Lab 1 — TA / Live-Coding Guide

A structured plan for conducting the Spring Boot E-Commerce application lab.

1. Lab Goal

Students build a small Spring Boot E-Commerce backend and understand the request flow from Controller → Service → Repository → Database and back.

The application uses Product as the main entity and implements four REST operations: retrieve all products, retrieve a product by ID, add a product, and update a product.

2. Initial Overview

Introduce Spring Boot as an annotation-driven framework that follows Convention over Configuration. Present the application request flow:

```
Client → Controller → Service → Repository → MySQL → Repository → Service → Controller → Client
```

Briefly introduce the roles of **pom.xml** for project dependencies and **application.properties** for application/database configuration.

3. Project Creation

Create the project using Spring Initializr with:

Item	Selection
Project	Maven
Language	Java
Spring Boot	4.1.1
Java	17
Dependencies	Spring Web, Spring Data JPA, MySQL Driver, Lombok, Springdoc OpenAPI

4. Database Setup

Use the MySQL command-line client and execute the prepared Lab 1 SQL script. The database is **ecommerceDB** and the table is **products**.

The table contains: id, name, description, price, category, and stock.

Configure the Spring Boot datasource in **application.properties** using the student's local MySQL credentials.

5. Application Construction

Build the application incrementally in the following order:

Stage	Component	Purpose
1	Product Entity	Map the Product class to the products table.
2	ProductRepository	Provide database access through JpaRepository<Product, Long>.
3	ProductService	Implement product operations and application logic.
4	ProductController	Expose the product operations as REST endpoints.

6. REST API Implementation

Method	Endpoint	Purpose	Response
GET	/api/products/get-all	Retrieve all products	200 OK

GET	/api/products/get/{id}	Retrieve one product	200 OK / 404 Not Found
POST	/api/products/add-product	Add a product	201 Created
PUT	/api/products/update/{id}	Update a product	200 OK / 404 Not Found

Use **ResponseEntity** to demonstrate status codes and response bodies. Use **@PathVariable** for the product ID and **@RequestBody** for POST/PUT request data.

7. Live Demonstration Flow

Introduce the architecture → create the Spring Boot project → configure the database → create Product → create ProductRepository → create ProductService → create ProductController → verify the first GET request → implement GET by ID → implement POST → implement PUT → trace the complete request flow → demonstrate Swagger if the application is complete.

8. API Verification

Use the browser for the GET endpoints and Postman for POST and PUT. The provided Postman collection contains the four Lab 1 requests.

Verify the database after POST and PUT so students can connect the REST operation with the persisted database state.

9. Git Recovery Checkpoints

Checkpoint	Application state
lab1-start	Initial Spring Boot project
lab1-db-config	Database configuration
lab1-entity	Product entity
lab1-repository	Product repository
lab1-get	Get-all API
lab1-get-by-id	Get-by-ID API
lab1-post	POST API
lab1-put	PUT API
lab1-complete	Complete Lab 1 application

Students who join late or need to recover can clone the repository, list the tags, and switch to the required checkpoint. Git supports switching to a tagged commit in detached HEAD mode for this type of recovery.
■cite■turn0search0■turn0search1■

```
Example: git switch --detach lab1-get
```

10. Final Lab State

The completed application contains the Product entity, repository, service, controller, MySQL integration, four REST endpoints, Postman collection, and Git recovery checkpoints. The final application is ready for the subsequent authentication-focused lab.